

Reviewer Pack — Schur-Weyl Duality Rank Bounds XOR Circuit Depth

Entry 5be7b1a76154 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:03:46 UTC

Schur-Weyl Duality Rank Bounds XOR Circuit Depth

Entry ID: 5be7b1a76154 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:03:46 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality and Algebraic Combinatorics **Field B** (complexity object): Boolean Circuit Complexity

Statement:

The rank of the Schur-Weyl duality representation $\rho(f)$ for a homogeneous polynomial f , when applied to XOR circuits with n inputs, bounds the depth of the circuit such that $D[\text{XOR}(n)] = \Theta(\rho(f)^2)$

Rationale (proposer's reasoning):

Schur-Weyl duality has been used to study representation theory in algebraic combinatorics. It provides a way to represent polynomials as linear combinations of lower-degree polynomials. This conjecture suggests that the structure preserved by Schur-Weyl duality could reveal hidden complexity in XOR circuits.

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cf8519cd6ab887e4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 30 random seeds, the correlation coefficient between circuit depth D and $\rho(f)^2$ is greater than or equal to 0.8 AND the mean difference between D and $\rho(f)^2$ is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_polynomial(n, d):
    coeffs = [random.randint(1, 10) for _ in range(d + 1)]
    x = 'x'
    return sum(c * eval(f'{x}**{i}') for i, c in enumerate(coeffs))

def generate_xor_circuit(depth, n):
    circuit = []
    for _ in range(depth):
        gate = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(n)]
        circuit.append((gate, inputs))
    return circuit

def evaluate_polynomial(poly, x_val):
    x = x_val
    return eval(poly)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            d = random.randint(1, n // 2)
            poly = generate_polynomial(n, d)
            depth = len(generate_xor_circuit(d, n))

            p_value = evaluate_polynomial(poly, 1) # Evaluate at x=1 for simplicity
            rho_squared = p_value ** 2

            results.append({
                "n": n,
                "d": d,
                "poly": poly,
```

```

        "depth": depth,
        "rho_squared": rho_squared
    })

    correlation_sum = 0
    diff_sum = 0
    instances_tested = len(results)

    for result in results:
        correlation_sum += result["depth"] / math.sqrt(result["rho_squared"])
        diff_sum += abs(result["depth"] - result["rho_squared"])

    if instances_tested < 30:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    correlation_mean = correlation_sum / instances_tested
    diff_mean = diff_sum / instances_tested

    return {
        "metric_name": "correlation",
        "metric_value": correlation_mean,
        "instances_tested": instances_tested,
        "conjecture_holds": correlation_mean >= 0.8 and diff_mean <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    correlation_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = len(correlation_values) / len(results)

    if not correlation_values:
        RESULT = "INCONCLUSIVE no_valid_data"
    elif support_fraction >= 0.8:
        RESULT = f"SUPPORTED mean={sum(correlation_values)/len(correlation_values):.2f} std={math.
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        RESULT = f"FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={first_fail

    print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_837c8b94.py", line 92, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_837c8b94.py", line 44, in run_trial
    poly = generate_polynomial(n, d)
           ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_837c8b94.py", line 21, in generate_poly
    return sum(c * eval(f'{x}**{i}') for i, c in enumerate(coeffs))
               ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_837c8b94.py", line 21, in <genexpr>
    return sum(c * eval(f'{x}**{i}') for i, c in enumerate(coeffs))
               ~~~~~~
```

```
File "<string>", line 1, in <module>
```

TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required number of trials to evaluate the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, rerun the test with at least 30 random seeds to properly assess the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11607
2	propose	ollama_remote	glm4:latest	0	0	10024
3	preregistration	ollama_remote	glm4:latest	0	0	5532
4	novelty	ollama_remote	glm4:latest	0	0	4922
5	novelty	ollama_remote	glm4:latest	0	0	5208
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34533
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7647
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9331
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10110

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	8519

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107433 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/5be7b1a76154.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5be7b1a76154.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5be7b1a76154.tar.gz (if generated)